Autobot_v1 Operations & Publishing Guide

Autobot_v1 Project

2026-08-19

Table of Contents

Practical, code-grounded guides for running, deploying, and distributing the Autobot_v1 EA. For the underlying strategy design, see `docs/superpowers/specs/2026-08-09-mt5-trend-ea-design.md`.

Read in this order:

1. Running the EA — install, compile, attach, full input reference, demo-only guard
2. Telegram Alerts — bot setup, chat ID, WebRequest allow-listing
3. VPS Deployment — MetaQuotes VPS vs third-party Windows VPS, 24/5 unattended running
4. Publishing to the MQL5 Market — seller side: submission, source protection, the demo-only decision
5. Subscribing and Running — buyer side: purchasing, activation, running a Market copy

A note that applies across all five guides

Autobot_v1 refuses to trade on a live account by default (`InpAllowLiveAccount=false`), a deliberate and verified design decision, not a bug. Every guide above treats that as a real constraint rather than something to route around.

Running Autobot_v1

Autobot_v1 is a multi-symbol H4-bias / H1-Donchian-breakout trend-following EA for XAUUSD, BTCUSD, and ETHUSD. See `docs/superpowers/specs/2026-08-09-mt5-trend-ea-design.md` for the full design spec.

Prerequisites

- MetaTrader 5 desktop terminal
- A **demo account** — the EA refuses to trade on a live account by default (see Demo-only guard below)
- Broker/symbols available: XAUUSD, BTCUSD, ETHUSD (or your broker's equivalent symbol names)

1. Install the files

Copy the EA folder into your terminal's data folder, preserving the `Include/` subfolder:

```
<Terminal Data Folder>/MQL5/Experts/Autobot_v1/Autobot_v1.mq5
<Terminal Data Folder>/MQL5/Experts/Autobot_v1/Include/*.mqh
```

Find your terminal's data folder from MT5: **File > Open Data Folder**.

2. Compile

1. Open `Autobot_v1.mq5` in MetaEditor (F4 from MT5, or double-click the file).
2. Press **Compile** (F7). Confirm zero errors.
3. Optional sanity check: the repo includes standalone test scripts under `MQL5/Scripts/Autobot_v1_Tests/`. Copy that folder into `MQL5/Scripts/` too, compile `SmokeTest_Compile.mq5`, and run it from the Navigator to confirm all modules link correctly before attaching the live EA.

3. Enable algo trading

In the MT5 toolbar, make sure **Algo Trading** is toggled on (green). The EA will not place orders otherwise.

4. Attach to charts

Attach **one instance per symbol** you want traded — open a chart for XAUUSD, BTCUSD, and/or ETHUSD and drag `Autobot_v1` onto each from the Navigator. The EA is single-symbol-per-chart; multi-symbol handling inside one instance is not how it's wired.

In the EA's **Common** tab, confirm **Allow WebRequest for listed URL** is available if you plan to use Telegram alerts (configured in the **Dependencies** tab — see Telegram Alerts).

Demo-only guard

By design, `OnInit()` refuses to run on a non-demo account:

> "Autobot_v1: refusing to run on a non-demo account (v1 is demo-only per design spec). Set InpAllowLiveAccount=true to override."

This was added and verified deliberately during development (see `.superpowers/sdd/progress.md`), not an oversight. `InpAllowLiveAccount` is a real

input you *can* set to `true` to override it — but do that only as your own informed decision after you've validated the strategy yourself; this guide does not recommend flipping it.

Input reference

Inputs are grouped in `MQL5/Experts/Autobot_v1/Include/Config.mqh`. Values shown are the shipped defaults.

Risk Management

Input	Default	Meaning
`InpRiskPercent`	1.0	Risk per trade, % of equity
`InpDailyLossPercent`	5.0	Daily loss circuit breaker, %
`InpMaxDrawdownPercent`	15.0	Max drawdown circuit breaker, %
`InpCorrelatedCapPercent`	1.5	Combined BTC+ETH open-risk cap, %
`InpClearMaxDrawdownBreaker`	false	Explicit human re-enable after a max-drawdown trip

Trading Logic

Input	Default	Meaning
`InpEMAPeriod`	200	H4 EMA period for trend bias
`InpDeadbandATRMultiplier`	0.1	Bias deadband, × H4 ATR
`InpDonchianPeriod`	20	H1 Donchian channel period
`InpATRPeriod`	14	ATR period (used on both H1 and H4)
`InpATRStopMultiplier`	2.0	Initial stop = entry ± N × ATR(H1)
`InpBreakevenBufferPoints`	20	Buffer added to breakeven SL, in points

Execution

Input	Default	Meaning
`InpMagicBase`	100000	Base magic number (per-symbol offset added)
`InpMaxRetries`	3	Max order-send retries on transient errors
`InpSlippagePointsGold`	50	Max deviation, XAUUSD
`InpSlippagePointsCrypto`	200	Max deviation, BTCUSD/ETHUSD
`InpMaxSpreadPointsGold`	50	Spread guard, XAUUSD

Input	Default	Meaning
`InpMaxSpreadPointsBTC`	3500	Spread guard, BTCUSD
`InpMaxSpreadPointsETH`	300	Spread guard, ETHUSD
`InpAllowLiveAccount`	false	Explicitly permit non-demo trading (see Demo-only guard)

Alerting

Input	Default	Meaning
`InpEnableTelegram`	false	Enable Telegram alerts
`InpTelegramBotToken`	“”	Telegram bot token (set per-deployment, never commit to source)
`InpTelegramChatID`	“”	Telegram chat ID
`InpHeartbeatHours`	24	Heartbeat notification interval, hours

Full setup for this section: Telegram Alerts.

Symbol Selection

Input	Default	Meaning
`InpTradeXAUUSD`	true	Allow new entries on XAUUSD
`InpTradeBTCUSD`	true	Allow new entries on BTCUSD
`InpTradeETHUSD`	true	Allow new entries on ETHUSD

Where the EA writes its state

Both files live in the terminal’s shared **Common** files folder (`FILE_COMMON`), not the per-terminal data folder — relevant if you ever run more than one terminal instance on the same machine (see VPS Deployment):

- `Autobot_v1_state.bin` — persisted equity baselines, circuit-breaker state, trail phase (survives terminal/VPS restarts)
- `Autobot_v1_log.csv` — trade/event log

Strategy Tester runs use separate `.TEST.*` filenames so backtests never read or corrupt live state.

Next steps

- Telegram Alerts — get notified of trades and heartbeats
- VPS Deployment — keep the EA running 24/5 without your own PC

Telegram Alerts

Autobot_v1 can push trade events, circuit-breaker trips, and a periodic heartbeat to Telegram via `Include/Notifier.mqh`. This is best-effort: a slow or failed Telegram call never blocks trade management, and alerts are automatically suppressed during Strategy Tester and optimization runs.

1. Create a Telegram bot

1. In Telegram, message **@BotFather**.
2. Send `/newbot` and follow the prompts (choose a name and a unique username ending in `bot`).
3. BotFather replies with a **bot token** — a string like `123456789:AAH...`. Keep it private.

2. Get your chat ID

1. Send any message to your new bot (or add it to a group/channel and send a message there).
2. In a browser, open:
   `https://api.telegram.org/bot<YOUR_BOT_TOKEN>/getUpdates`
3. Find `"chat":{"id": ... }` in the JSON response — that number is your `InpTelegramChatID`. For a group chat this is typically negative.

3. Allow-list the Telegram API URL in MT5

MT5 blocks all `WebRequest` calls to URLs not explicitly allow-listed, per terminal install:

1. In MT5: **Tools > Options > Expert Advisors**.
2. Check **Allow WebRequest for listed URL**.
3. Add: `https://api.telegram.org`
4. Click OK.

Do this on every terminal instance that will run the EA — the allow-list is per-terminal, not inherited from source, so it must be repeated on a VPS install too.

4. Configure the EA inputs

When attaching (or re-attaching) `Autobot_v1` to a chart, set on the **Inputs** tab under **Alerting**:

Input	Value
`InpEnableTelegram`	`true`
`InpTelegramBotToken`	your bot token
`InpTelegramChatID`	your chat ID
`InpHeartbeatHours`	how often you want a “still alive” ping (default 24)

Never hardcode the token/chat ID into the `.mq5` source — set them per-deployment via the input dialog, so a shared/published copy of the EA doesn't leak your credentials.

5. Verify it works

- Trigger a manual test: temporarily set `InpHeartbeatHours` to 1 and wait, or just leave the EA running past its next heartbeat interval, and confirm a message like *"Autobot_v1: heartbeat - EA is running."* arrives.
- If nothing arrives, check the **Experts** log tab for: `Notifier: Telegram WebRequest failed, error <code> (is the URL allow-listed in Tools>Options>Expert Advisors?)` This means step 3 wasn't applied to this specific terminal.

What gets sent

- Circuit-breaker trips (daily loss, max drawdown)
- Order-send failures and `PositionModify` failures
- Periodic heartbeat (interval controlled by `InpHeartbeatHours`)

Push notifications to the MetaTrader mobile app (`SendNotification`) also fire alongside Telegram, independent of `InpEnableTelegram`, if you've linked a MetaQuotes ID in the terminal — Telegram is additive, not a replacement.

VPS Deployment

Running Autobot_v1 on a VPS keeps it trading 24/5 without depending on your own PC staying on and connected. Two paths: MetaQuotes' built-in VPS, or a general-purpose Windows VPS.

Option A: MetaQuotes VPS (simplest)

1. In MT5, with the EA already attached and running on your own machine: **right-click the chart > Trading > VPS Hosting** (or the *File > Open an Account* VPS prompt).
2. Choose a VPS location close to your broker's server for lower latency.
3. Rent the VPS. MT5 migrates your currently-running terminal state — including the attached EA and its chart/inputs — to the VPS.
4. Confirm on the VPS view that **Algo Trading** is enabled and the EA shows in the top-right corner without a red "X" (no errors).

This is the lowest-effort option since MetaQuotes handles the OS/terminal setup for you.

Option B: Third-party Windows VPS

Use this if you want more control (e.g. running other software, or a specific region MetaQuotes doesn't offer).

1. Provision a Windows Server/Windows VPS from any provider. Minimum spec: this EA is lightweight (5-second timer, a handful of indicator handles per symbol) — a small instance is sufficient.

2. RDP into the VPS.
3. Install the MT5 terminal for your broker (download from the broker's site, not a generic MetaQuotes installer, to ensure it connects to the right trade servers).
4. Log in with your **demo account** credentials (see the demo-only guard in Running Autobot_v1).
5. Repeat the install steps from Running Autobot_v1: copy `Autobot_v1.mq5` + `Include/`, compile, enable Algo Trading, attach to each symbol chart.
6. If using Telegram alerts, repeat the WebRequest allow-listing from Telegram Alerts on this terminal — it is a per-install setting, not something that transfers automatically.
7. Set the terminal to auto-login and auto-start on VPS reboot: **Tools > Options > General** and enable auto-login, and place a shortcut to `terminal64.exe` in the Windows Startup folder so it survives a VPS reboot without you RDP'ing back in.

Things specific to this EA on a VPS

- **One terminal instance per machine.** State (`Autobot_v1_state.bin`) and the trade log (`Autobot_v1_log.csv`) are written to the terminal's shared **Common** files folder (`FILE_COMMON`), not per-terminal-data-folder. Running two separate terminal installs under the same Windows user on the same VPS would have them silently share/clobber the same state file. Stick to one terminal install per VPS.
- **After any VPS crash/reboot/interruption**, re-open the chart and confirm the EA reloaded correctly — persistence is designed to survive this (equity baselines, circuit-breaker trip flags, trail phase are all restored from disk on `OnInit`), but it's worth a manual check the first few times, especially confirming the daily-loss/max-drawdown breaker state matches what you expect.
- **Time zone**: the daily-loss breaker resets on the broker's trade-server day (`TimeTradeServer()`), not the VPS's local clock — no action needed, just don't be surprised if "daily" resets don't line up with your own timezone.
- **Telegram heartbeat is your VPS health check** — if `InpEnableTelegram` is on, a missed heartbeat (per `InpHeartbeatHours`) is a good signal something died (VPS down, terminal crashed, disconnected from broker).

Verifying it's actually running unattended

- Close your RDP session (don't shut down the VPS) and confirm via Telegram heartbeat, or by reconnecting later, that the EA kept running.
- Check the **Experts** and **Journal** tabs after a reconnect for any errors that occurred while you were away.

Publishing to the MQL5 Market

This covers publishing Autobot_v1 as a product on the MQL5 Market (mql5.com), from the seller side. MQL5.com's own submission checklist is the source of truth for exact current

requirements — treat this as an orientation, not a substitute for it, since Market policies do change over time.

Before you publish: the demo-only guard

Autobot_v1 currently **refuses to trade on a live account** by default (`OnInit` in `Autobot_v1.mq5`, gated by `InpAllowLiveAccount` — see Running Autobot_v1). This was a deliberate, verified design decision, not an oversight.

That has a direct consequence for Market publishing: most Market buyers expect to eventually run a product on a live account. You have two honest options, and this guide isn't going to pick one for you:

1. **Publish as-is, listed clearly as demo-only** (e.g. free, or as an "evaluate on demo" product). Buyers can set `InpAllowLiveAccount=true` themselves after their own evaluation, same as you would.
2. **Deliberately decide to relax the guard** for the published version, as a separate, conscious change — not something to do casually while writing docs. If you go this route, treat it as a real code change with its own review, not a docs edit.

Whichever you choose, don't let the product page imply live-readiness that the code doesn't actually have.

1. Seller account

1. Create/verify a seller account on mql5.com (a MetaQuotes ID account with seller status enabled — requires identity verification for withdrawals if you plan to charge money).
2. Free products typically have a lower bar than paid ones; if this is your first listing, consider starting free.

2. Prepare the product

1. Compile a clean, warning-free `.ex5` (MQL5 Market re-compiles/verifies your source server-side during submission — it does not accept a pre-compiled binary you upload standalone).
2. Write a clear product description: what it trades (XAUUSD/BTCUSD/ETHUSD), the strategy (H4 trend bias + H1 Donchian breakout), the full input reference (mirror the table in Running Autobot_v1), and — explicitly — that it ships demo-only unless the buyer opts in.
3. Prepare screenshots: EA attached to a chart, Strategy Tester results, and the Inputs dialog.
4. Have a Strategy Tester report ready (MQL5 Market screening includes automated testing on MetaQuotes' own infrastructure) — the `.superpowers/sdd/progress.md` record of a clean 2026-07-01→2026-08-08 BTCUSD/H1 tester run is the kind of evidence to keep on hand, though you'll want a fresh run against current market data before submitting.

3. Source protection

Since you're not distributing raw `.mq5` source:

- **MQL5 Cloud Protector** is automatic for anything sold through the Market — MetaQuotes applies asymmetric encryption and account/hardware binding server-side. No extra work required for Market-only distribution.
- If you also plan to distribute outside the Market (e.g. direct sales), see `.agents/skills/mql-developer/references/security-licensing.md` for custom account-based licensing and manual Cloud Protector usage.
- Either way: never hardcode a Telegram token, license key, or any other secret into the `.mq5` source before submission — those stay as empty-default `input` fields the buyer fills in themselves, exactly as `InpTelegramBotToken`/`InpTelegramChatID` already are.

4. Submit for review

1. Upload via the MQL5 Market seller dashboard on mql5.com.
2. Automated screening checks for things like: no external DLL calls, no undisclosed network calls (your Telegram `WebRequest` call is fine since it's user-configured and off by default via `InpEnableTelegram=false`), and that it runs cleanly on MetaQuotes' own demo/tester environment.
3. Expect a review cycle — MetaQuotes staff may reject with specific feedback; address and resubmit.
4. Set pricing (or free), rental options if desired, and publish once approved.

5. After publishing

- Keep a versioned changelog; the Market shows version history to buyers, and `#property version` in `Autobot_v1.mq5` should be bumped on every update.
- Monitor product-page comments/support requests — this is buyer-facing once listed, unlike the internal `.superpowers/sdd/` task tracking used during development.

Next

For the buyer's side of this same product: Subscribing and Running (Buyer Side)

Subscribing and Running (Buyer Side)

This is the flow for someone who found Autobot_v1 on the MQL5 Market (as opposed to building it from this repo's source — see Running Autobot_v1 for that path).

1. Get the product

1. In the MT5 terminal: open the **Market** tab in the Toolbox (bottom panel), or browse mql5.com's Market section in a browser and open the product page.

2. Depending on how the seller listed it, choose **Free**, **Rent** (time-limited), or **Buy** (permanent license).
3. Confirm the purchase/activation in your MQL5.com account if prompted.

2. Download into your terminal

1. Back in MT5's **Market** tab, find the product under **Purchased**.
2. Click **Download** (or it may auto-sync on next terminal restart). This places the compiled `.ex5` directly into your terminal's `MQL5/Experts/` folder — no MetaEditor/compiling needed on the buyer side, since Market products ship pre-compiled and protected.

3. Attach and activate

1. Open a chart for a symbol the EA trades (XAUUSD, BTCUSD, or ETHUSD — check the product description for which symbols/names your broker uses).
2. Drag the EA from the Navigator onto the chart.
3. On first run, MT5 may prompt an **activation** step tied to your MQL5 account/account number (this is the Market's built-in licensing, separate from anything in the EA's own inputs).
4. Make sure **Algo Trading** is enabled in the toolbar.

4. Configure inputs

Same input set as the source-build path — see the tables in Running Autobot_v1. In particular:

- Confirm you're on a **demo account** unless you've made your own informed decision to enable `InpAllowLiveAccount` (read the product description's stance on this — see the callout in Publishing to the MQL5 Market).
- If you want Telegram alerts, follow Telegram Alerts — this works identically whether you're running the source build or a Market copy, since the bot token/chat ID are input fields you fill in yourself, not something the seller can see or set for you.

5. Running unattended

Nothing here differs from the source-build path — see VPS Deployment. One Market-specific note: an "activated"/purchased EA is typically tied to a limited number of accounts (check the product's license terms) — moving to a VPS may count as a new activation, so check the seller's terms on activation limits before deploying to a VPS if you're on a paid license.

Troubleshooting

Symptom	Likely cause
EA won't start, log shows a "demo-only" refusal message	Working as designed — see the demo-only guard note above; this is not a bug

Symptom	Likely cause
No Telegram messages	WebRequest URL not allow-listed on *this* terminal, or `InpEnableTelegram=false` — see Telegram Alerts
“Invalid account” / activation prompt loops	Check the Market license’s activation limit for your purchase type
EA attached but never trades	Confirm Algo Trading is on, confirm the chart symbol matches one of `InpTradeXAUUSD`/`InpTradeBTCUSD`/`InpTradeETHUSD`, and check spread against the relevant `InpMaxSpreadPoints*` guard